

Introduction to Neural Networks

Previous session

In the previous session, you learned about:

- **History and Evolution** of Deep Learning
- Importance of **computational efficiency** in deep learning
- Overview of **Machine Learning (ML)** and **Deep Learning (DL)**
- Difference between ML and DL with examples

Contents

Introduction to Neural networks (The fundamental building block of Deep Learning)

- Biological inspiration for Neural Networks,
- Mathematical representation of Neuron,
- Perceptron and basic components of perceptron,
- Perceptron Learning Model,
- Types of Perceptron,
- Single Layer Perceptron numerical problems,
- Limitations of Single Layer Perceptron.

Biological Inspiration for Neural Networks

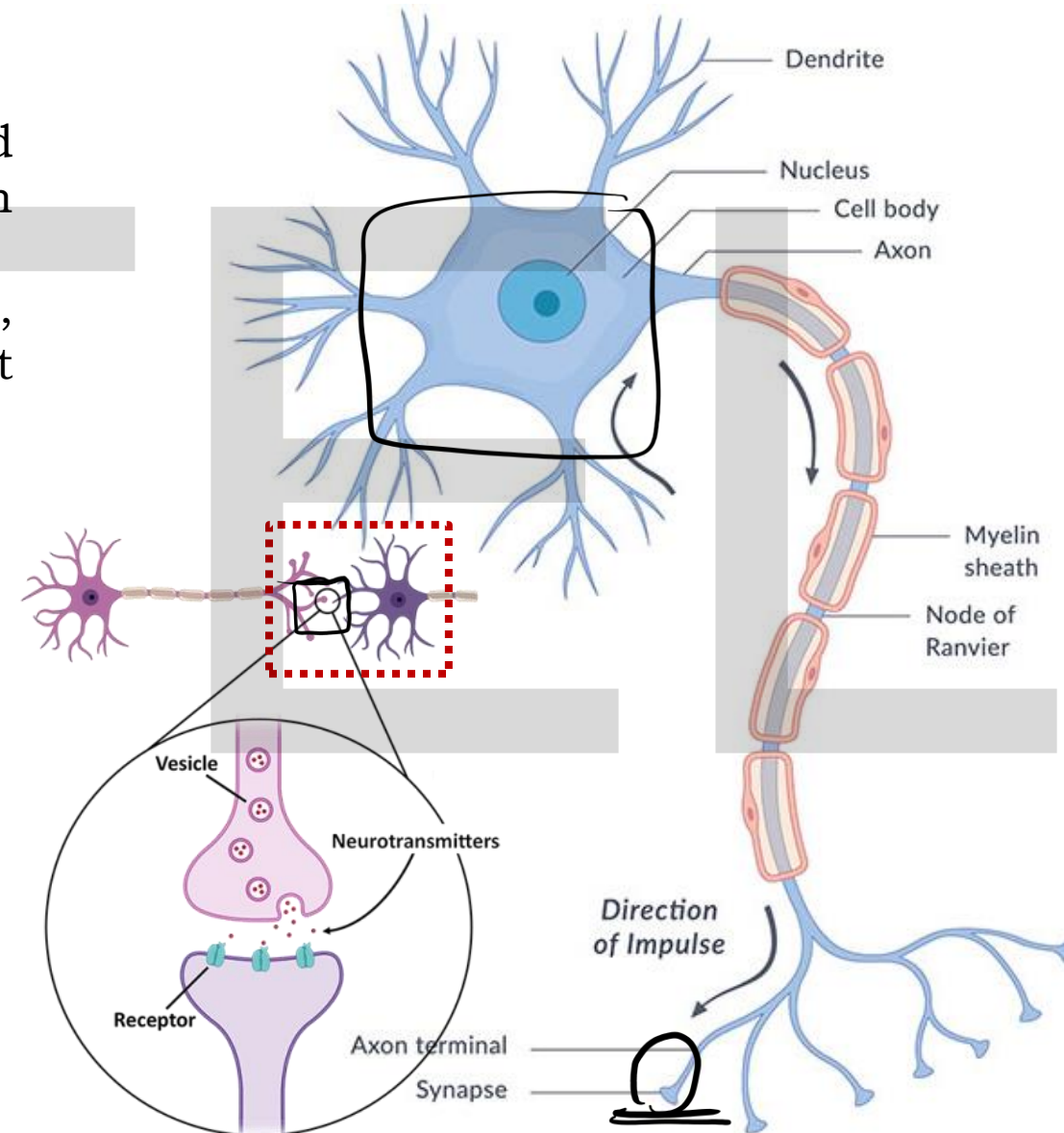
Human Brain as Inspiration

- The human brain consists of billions of interconnected neurons that process and transmit information through electrochemical signals.
- Each neuron receives inputs through dendrites, integrates them in the cell body, and sends an output signal through the axon to other neurons via synapses.

Axon: Thin fiber-like projection of a neuron. This axon carry the electrical signal and sends information to another neurons.

Synapses: A synapse is a connection point between the axon terminal of one neuron and the dendrite of another neuron.

The electrical signal from the axon is converted into a chemical signal that crosses the small synaptic gap.



Biological Inspiration for Neural Networks

Biological Neuron

Dendrites

Cell body

Firing

Axon

Synapse

Artificial Neuron

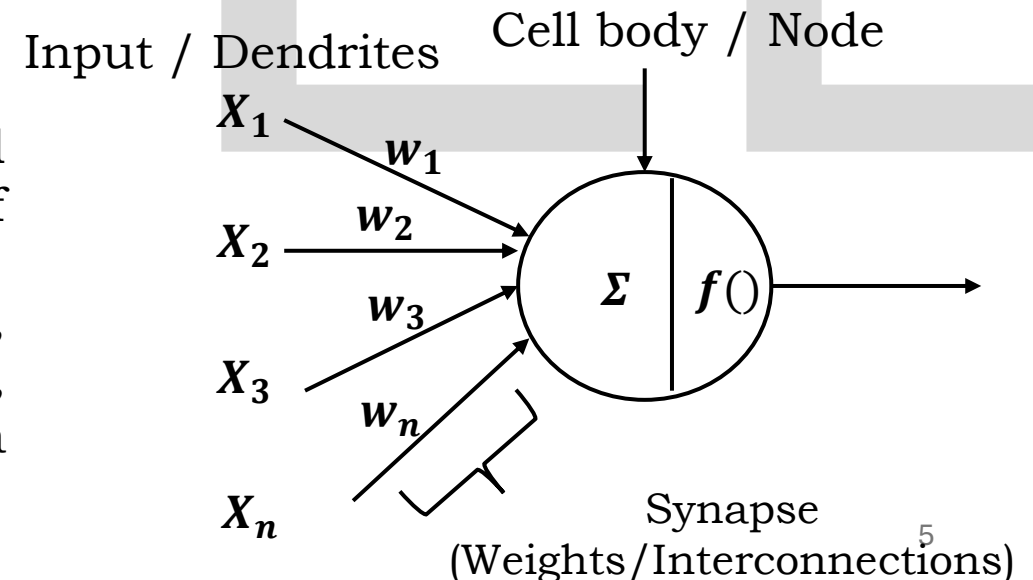
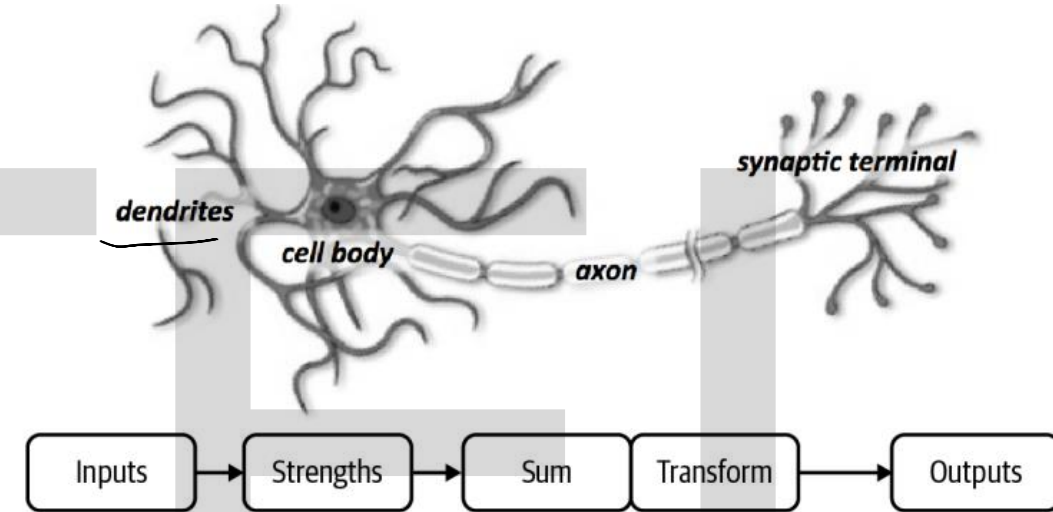
Inputs

Node / Summation unit

Activation Function

Connection/Output

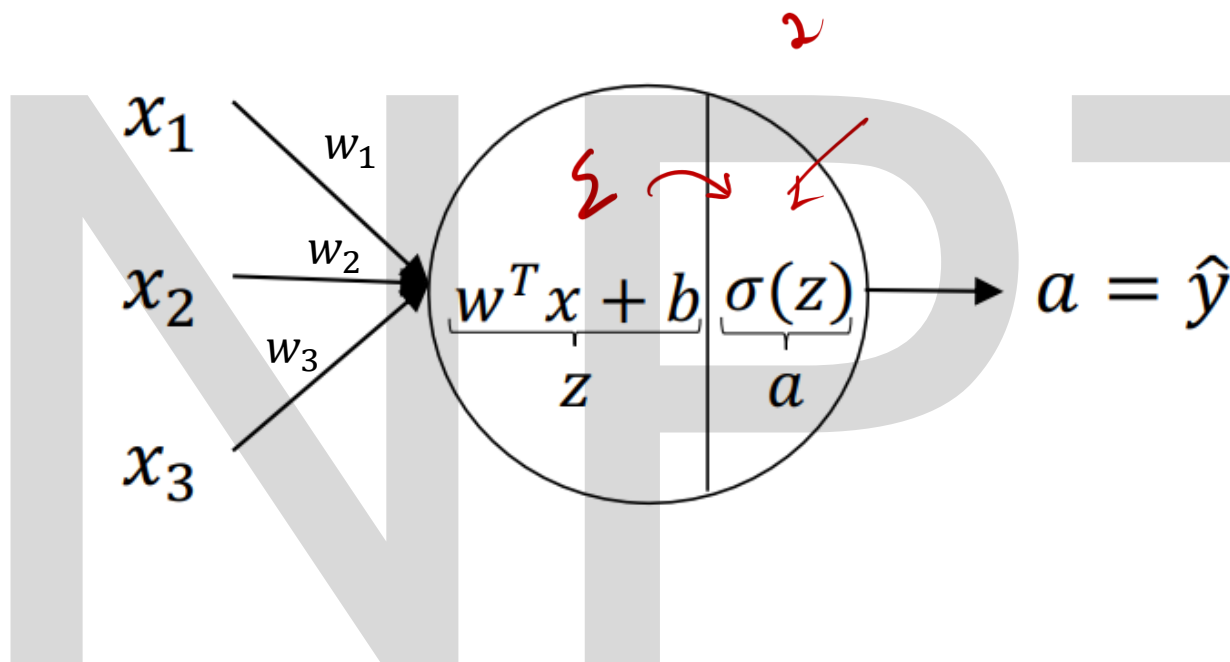
Weights



Artificial Analogy

- Artificial Neural Networks (ANNs) are computational models inspired by the structure and functioning of biological neurons.
- Each artificial neuron receives multiple inputs, multiplies them by their connection weights, sums them, and applies an activation function — mimicking how a biological neuron “fires” when sufficiently stimulated.

Mathematical Representation of Neuron



Suppose you're predicting if a student will pass based on **3 features**:

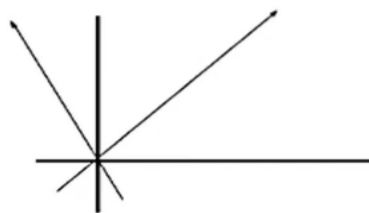
x_1 : Hours studied,
 x_2 : Attendance percentage,
 x_3 : Previous grade

Lets say: $x = \begin{bmatrix} 8 \\ 90 \\ 75 \end{bmatrix}$, $w = \begin{bmatrix} 0.4 \\ 0.3 \\ 0.2 \end{bmatrix}$, $b = 1$

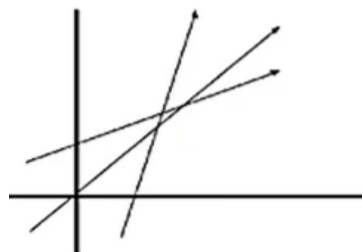
$$z = w^T x + b$$

$$a = \sigma(z)$$

Then: $x \cdot w$ is not possible. It has to be $w^T \cdot x$



Without bias



With bias

Perceptron

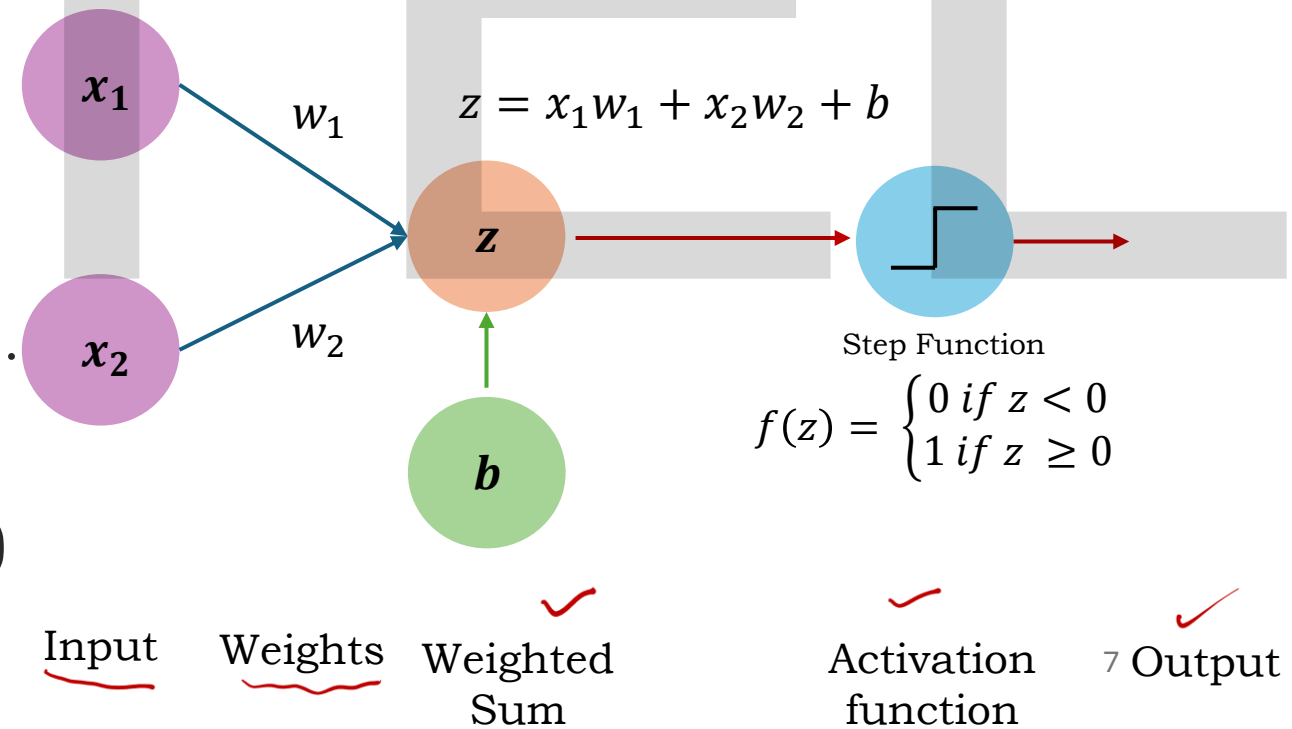
- A **Perceptron** is the simplest and oldest form of a neural network, introduced by Frank Rosenblatt in 1958.
- It is an algorithm used for **supervised learning of binary classifiers** — that is, it can classify data into **two classes** (e.g., 0 or 1, Yes or No, True or False).
- A perceptron consists of an **input layer** and an **output layer** (no hidden layers). It is mainly used to model simple **logical operations** such as **AND, OR, and NOR**, which have **binary outputs**.

Working of Perceptron:

Step 1: Multiply each input x_i by its corresponding weight w_i and add the bias b .

Step 2: Compute the Weighted Sum z .

Step 3: Pass the weighted sum through an activation function (usually a step function) to decide the output.



Example

Component	Example 1 – Spam Detection							
Input Features	x1= number of suspicious words; x2= % capital letters in subject; x3= number of links	$B < 0 \rightarrow$ spam $\downarrow$ not a spam <table><tr><td>x1</td><td>x2</td><td>x3</td></tr><tr><td>4</td><td>0.2</td><td>2</td></tr></table>$\rightarrow 1$ Input Values $x_1 = 4, x_2 = 0.2, x_3 = 2$	x1	x2	x3	4	0.2	2
x1	x2	x3						
4	0.2	2						
Weights	w1=0.8, w2=0.5, w3=0.3 ✓							
Bias	b=0.5 The bias helps shift the decision threshold so that the neuron can classify emails correctly even when all feature values are low.							
Summation Function	$S = (4 \cdot 0.8) + (0.2 \cdot 0.5) + (2 \cdot 0.3) + 0.5 = 4.4$							
Activation Function	Threshold $\theta = 3.0$ ✓ If $S \geq 3.0 \rightarrow$ Output = <u>1</u> (Spam), else 0 (Not Spam)	$x_1 w_1 + x_2 w_2 + x_3 w_3 + b$ $= 4.4$ $\theta = 3.0$ step Sp						
Output	1 (Spam) Interpretation: Since $S = 4.4 > 3.0$, the email is classified as Spam (1) . This means the combination of features (many suspicious words and links) strongly indicates spam behavior.							
Learning Algorithm	If spam email is misclassified as “Not Spam,” increase w1,w2,w3 and bias slightly.							

Perceptron learning algorithm

2. Repeat for each epoch:

For each training sample

1. Compute the weighted sum:

$$\text{sum} = \sum_{i=1}^n w_i^T \cdot x_i$$

2. Pass the weighted sum to the activation function

$$y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases}, \theta \text{ is threshold}$$

3. Calculate error: *target* - *computed*

4. Update the weights and bias

$$w_{\text{new}} \leftarrow w_{\text{old}} + \Delta w$$

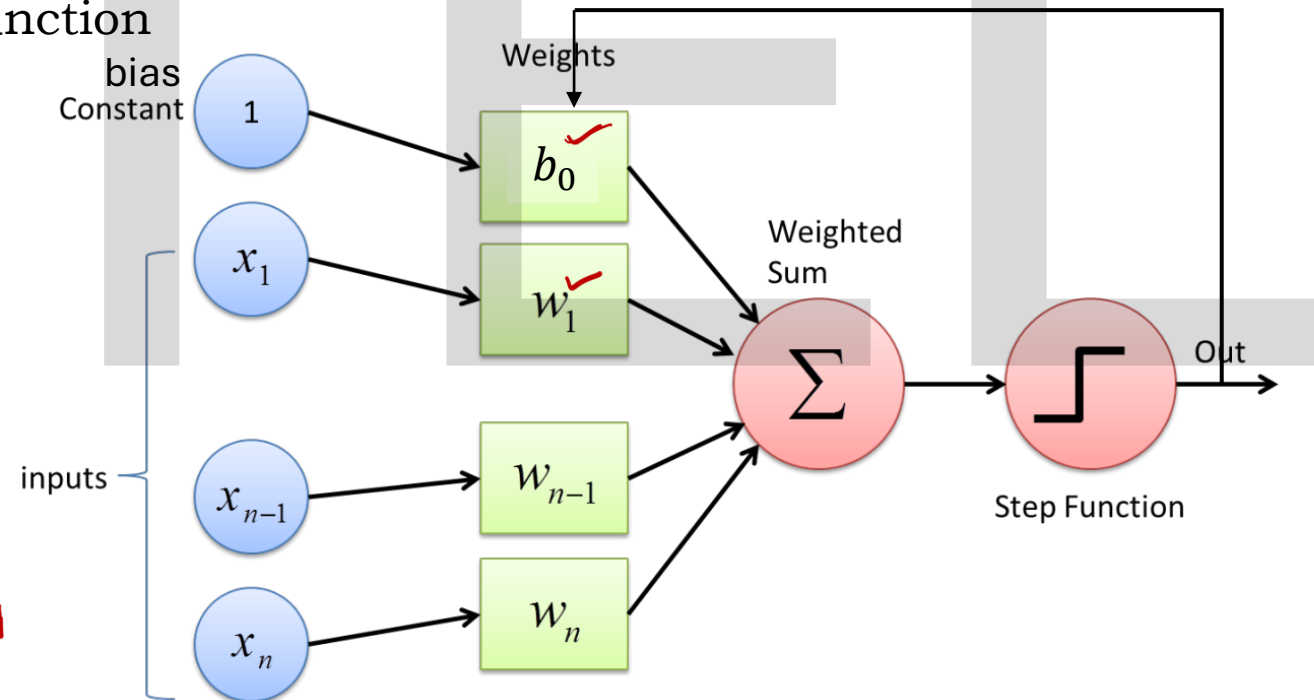
$$\Delta w = \eta (\text{target} - \text{computed}) x_i$$

$$\text{bias}_{\text{new}} \leftarrow \text{bias}_{\text{old}} + \eta (\text{target} - \text{computed})$$

1.Initialize:

1. Set weights $w_1, w_2, \dots, w_n$ and the bias b_0 to the random values.

2. Choose the learning rate η



Perceptron learning algorithm

2. Repeat for each epoch:

For each training sample

1. Compute the weighted sum:

$$\text{sum} = \sum_{i=1}^n w_i^T \cdot x_i$$

2. Pass the weighted sum to the activation function

$$\gamma = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases}, \theta \text{ is threshold}$$

3. Calculate error: *target* – *computed*

4. Update the weights and bias

$$w_{\text{new}} \leftarrow w_{\text{old}} + \Delta w$$

$$\Delta w = \eta (\text{target} - \text{computed}) x_i$$

$$\text{bias}_{\text{new}} \leftarrow \text{bias}_{\text{old}} + \eta (\text{target} - \text{computed})$$

1. Initialize:

1. Set weights $w_1, w_2, \dots, w_n$ and the bias b_0 to the random values.

2. Choose the learning rate η

Directly tells us the direction of error:

If the error is too small, increase the weight. ✓

If the error is too large, decrease the weight. ↓

This rule is **not derived from gradient descent** — it is a **heuristic update rule designed for linearly separable problems.**

Numerical Example 1: Single Layer Perceptron

Inputs: $x = [x_1, x_2] = [1.0, 2.0]$

Target: $y = 1$

Weights: $w = [w_1, w_2] = [0.2, -0.5]$

Bias: $b = 0.1$

Learning rate $\eta = 0.1$

Prediction rule if computed value is ≤ 0 then 0 otherwise 1

Lets solve:

$$\begin{aligned} w_{\text{new}} &= w_{\text{old}} + \Delta w \\ \Delta w &= \eta (\text{error}) x_1 \\ &= 0.1 (1) (1) \\ \Delta w &= 0.1 \end{aligned}$$

$w_1 = 0.3$

$$w_2 = -0.5 + 0.2 = -0.3$$

$$\Delta w = 0.1 (1) (2)$$

$$\Delta w_2 = 0.2$$

$$w_2 = -0.3$$

$$\begin{aligned} b &= b_{\text{old}} + \eta (\text{error}) \\ &= 0.1 + 0.1 (1) \end{aligned}$$

$$b = 0.2$$

$$\begin{array}{c|c} z & f \\ \hline \sum & \end{array} \rightarrow 0$$

$$\begin{aligned} z - c \\ 1 - 0 &= 1 \end{aligned}$$

$$\text{error} = 1$$

$$\begin{aligned} Z &= x_1 w_1 + x_2 w_2 + b \\ &= 1(0.2) + 2(-0.5) + 0.1 \\ &= 0.2 - 1 + 0.1 \end{aligned}$$

$$Z = -0.7$$

$$\hat{y} = 0$$

$$\begin{aligned} Z &\leq 0 = 0 \\ Z &> 0 = 1 \end{aligned}$$

$$x_1, x_2 = [1, 2]$$

$$w = [0.4, -0.1]$$

$$b = 0.3$$

$$Z = x_1 w_1 + x_2 w_2 + b$$

$$= 1(0.4) + 2(-0.1) + 0.3$$

$$= 0.4 - 0.2 + 0.3$$

$$Z = 0.5 \quad \hat{y} = 1$$

$$\text{Error} = t - c$$

$$= 1 - 1$$

$$\boxed{\text{Error} = 0}$$

$$w^T x + b$$

$$x_1 w_1 + x_2 w_2 + b$$

$$x_1(0.4) + x_2(-0.1) + 0.3$$

$$\boxed{0.4x_1 - 0.1x_2 + 0.3 = 0}$$

$$x_1 = 2 \quad x_2 = 1 \leftarrow \text{Testing}$$

$$0.4(2) - 0.1(1) + 0.3$$

$$Z = 0.8 - 0.1 + 0.3$$

$$\boxed{Z = 1} > 0 \quad \hat{y} = 1$$

1	2	1
2	1	-?
0	5	0

$$x_1 = 0, x_2 = 5$$

$$Z = 0.4(0) - 0.1(5) + 0.3$$

$$Z = -0.2 < 0$$

$$\hat{y} = 0$$

Types of Perceptron

1. Single Layer Perceptron model: One of the simplest forms of Artificial Neural Networks (ANN), limited to learning linearly separable patterns.

The **architecture** consists of **only an input layer and an output layer** (with no hidden layers).

The **activation function** used is typically a **step or threshold function**. It is more suitable for **binary classification (1 or 0)**.

The model **uses the perceptron learning rule and does not employ backpropagation**.

Update the weights and bias

$$w_{new} \leftarrow w_{old} + \Delta w$$

$$\Delta w = \eta(\text{target} - \text{computed})x_i$$

$$\text{bias}_{new} \leftarrow \text{bias}_{old} + \eta(\text{target} - \text{computed})$$

Cannot handle non-linearly separable data